

Q and A – Lecture 2

Algorithms and Complexity

Prof. Noel Fernando

Question 1

- Find the running time of $f(n) = 3n^2 + 5n + 20$

Question 1-Find the running time of $f(n) = 3n^2 + 5n + 20$

- Question1 : Polynomial Function
- $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$$f(n) = 3n^2 + 5n + 20$$

$$\text{Choose } g(n) = n^2$$

$$\text{For } c=4, n_0=10$$

$$3n^2 + 5n + 20 \leq 4n^2 \text{ for all } n \geq 10$$

$$\text{Therefore, } f(n) \in O(n^2)$$

Questions (2 and 3)

2. Using a suitable example, show the worst-case running time of an if-then-else statement.
3. Using a suitable example, show the worst-case running time of a simple loop program segment

Answer - Question 2 :“if-else” Running Time

The running time of an if-else

```
if (cond) S1  
else S2
```

is never more than: $RT_{\text{worst-case}} = RT(\text{cond}) + \max\{RT(S1), RT(S2)\}$

is never less than: $RT_{\text{best-case}} = RT(\text{cond}) + \min\{RT(S1), RT(S2)\}$

Answer - Question 3: “for-loop” Running Time

The running time of a simple loop

```
for (int i=0; i<n; i++) {  
    body; }
```

is: $RT(\text{loop}) * RT(\text{body}) = n * RT(\text{body})$

Question 4

- Consider the following C program segment.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 10; // example input size
```

```
    printf("Hello\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

```
}
```

Calculate the worst-case time complexity of the above code.

Answer – Question 4

```
#include <stdio.h>
int main() {
    int n = 10; // example input size
    printf("Hello\n");
    for (int i = 0; i < n; i++)
        printf("%d\n", i);
    return 0;
}
```

- **Step 1: Identify Basic Operations**
- `int n = 10;` → assignment → **$O(1)$**
- `printf("Hello\n");` → print statement → **$O(1)$**
- `for (int i = 0; i < n; i++) printf("%d\n", i);` → loop executes **n times**, each iteration has **$O(1)$**
- print operation → **$O(n)$**
- **Step 2: Sum All Operations**
- $T(n) = O(1) + O(1) + O(n) = O(n)$
- **Step 3: Worst-Case Analysis**
- The loop runs exactly **n times** regardless of input (no conditional branches).
- Therefore, **worst-case running time** is dominated by the loop → **$O(n)$**

Question 5 - Calculate the worst-case running time of the following program

Multiply for Nested Loops

Python code

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

C code

```
#include <stdio.h>  
int main() {  
    int n = 5; // example input size  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            printf("%d %d\n", i, j);  
}  
return 0;  
}
```

Answer – Question 5

- Step 1: Identify the Loops and Basic Operations
- **Outer loop:** for (int i = 0; i < n; i++) → runs **n times** → **O(n)**
- **inner loop:** for (int j = 0; j < n; j++) → runs **n times for each outer loop iteration** → **O(n)**
- **Print statement inside inner loop:** printf("%d %d\n", i, j) → **O(1)** per iteration
- Step 2: Calculate Total Running Time
- Total iterations of inner loop = **n × n = n²**
- Each iteration does **O(1)** work
- $T(n) = n \times n \times O(1) = O(n^2)$

$$T(n) \in O(n^2)$$

```
#include <stdio.h>
int main() {
    int n = 5; // example input size
    for (int i = 0; i < n; i++) {    // Outer loop O(n)
        for (int j = 0; j < n; j++) { // Inner loop O(n)
            printf("%d %d\n", i, j); // O(1)
        }
    }
    return 0;
}
```

Question 6 - Calculate the worst-case running time of the following program

Keep the Dominant Term

When there are multiple terms, keep the *highest order*:

Python Code

```
for i in range(n):
    for j in range(n):
        print(i, j)
for k in range(n):
    print(k)
```

C code

```
#include <stdio.h>

int main() {
    int n = 5; // example input size
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d %d\n", i, j);
        }
    }
    // Separate loop: O(n)
    for (int k = 0; k < n; k++) {
        printf("%d\n", k);
    }
    return 0;
}
```

Question 6 - Calculate the worst-case running time of the following program

Keep the Dominant Term

When there are multiple terms, keep the *highest order*:

Python Code

```
for i in range(n): # O(n)
    for j in range(n): # O(n)
        print(i, j) # O(1)
for k in range(n): # O(n)
    print(k)
```

C code

```
#include <stdio.h>

int main() {
    int n = 5; // example input size
    // Nested loops: O(n) * O(n) = O(n²)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d %d\n", i, j);
        }
    }
    // Separate loop: O(n)
    for (int k = 0; k < n; k++) {
        printf("%d\n", k);
    }
    return 0;
}
```

Answer : Question 6

Keep the Dominant Term

When there are multiple terms, keep the *highest order*:

Python Code

```
for i in range(n): # O(n)
    for j in range(n): # O(n)
        print(i, j) # O(1)
for k in range(n): # O(n)
    print(k)
```

- **Time Complexity**

Nested loops: $O(n^2)$
Separate loop: $O(n)$
Together: $O(n^2) + O(n)$
Dominant term: $O(n^2)$

C code

```
#include <stdio.h>

int main() {
    int n = 5; // example input size
    // Nested loops:  $O(n) * O(n) = O(n^2)$ 
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d %d\n", i, j);
        }
    }
    // Separate loop:  $O(n)$ 
    for (int k = 0; k < n; k++) {
        printf("%d\n", k);
    }
    return 0;
}
```

Question Number 7: Find the running time of the recursive factorial

$\text{fact}(n) = n * \text{fact}(n-1) = n * (n-1) * \text{fact}(n-2) = \dots = n * (n-1) * (n-2) * \dots * \text{fact}(1) = n!$

```
public int fact(int n) {  
    if n<=1 return 1;  
    else return n*fact(n-1);  
}
```

Answer – Question 7

$\text{fact}(n) = n * \text{fact}(n-1) = n * (n-1) * \text{fact}(n-2) = \dots = n * (n-1) * (n-2) * \dots * \text{fact}(1) = n!$

```
public int fact(int n) {  
    if n<=1 return 1;  
    else return n*fact(n-1);  
}
```

- $\text{RT}(1) = C$ (condition evaluation + assignment)
- $\text{RT}(2) = C + \text{RT}(1) = 2C$ (condition evaluation + execution of $\text{fact}(1)$)
- $\text{RT}(3) = C + \text{RT}(2) = 3C$ (condition evaluation + execution of $\text{fact}(2)$)
- ...
- $\text{RT}(n) = \text{RT}(n-1) + C = C + (n-1)C = nC$

$$\text{RT}(n) = O(n)$$

Analysis Of Recursive Factorial method

- Example1: Form and solve the recurrence relation for the running time of factorial method and hence determine its big-O complexity:

```
long factorial (int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial (n - 1);
}
```

$$T(0) = c \quad (1)$$

$$T(n) = b + T(n - 1) \quad (2)$$

$$= b + b + T(n - 2) \quad \text{by substituting } T(n - 1) \text{ in (2)}$$

$$= b + b + b + T(n - 3) \quad \text{by substituting } T(n - 2) \text{ in (2)}$$

...

$$= kb + T(n - k)$$

The base case is reached when $n - k = 0 \rightarrow k = n$, we then have:

$$T(n) = nb + T(n - n)$$

$$= bn + T(0)$$

$$= bn + c$$

Therefore the method factorial is $O(n)$

Question 8 – Calculate the Running time of the following program segment.

```
int k=0;
for (int i=0; i<n; i++) {                               /* outer_loop */
    if even(i) { for (int j=0; j<n; j++)                 /* inner_loop */
                  k++; };
    else k *= 2;
};
```

Answer question 8

```
int k=0;
for (int i=0; i<n; i++) {                               /* outer_loop */
    if even(i) { for (int j=0; j<n; j++)                 /* inner_loop */
        k++; };
    else k *= 2;
};
```

$$RT(n) = c + RT(\text{outer_loop}) * RT(\text{body}) = c + n * RT(\text{body})$$


$$RT_{\text{worst-case}}(\text{body}) = c + \max\{ RT(\text{inner_loop}), c \} = c + n*c$$

$$RT(n) \leq n * RT_{\text{worst-case}}(n) = c + n * (c + n * c) \Rightarrow RT(n) = O(n^2)$$

Activa
Go to Se

Question 9

- Consider an algorithm whose running time is given by $T(n) = n - 1$. Using Big-O notation, determine the time complexity of the algorithm and explain why it is classified as linear

Answer – Question 9

- we looked at an algorithm with complexity $T(n) = n - 1$.
- Using Big-O notation this can be written as $T(n) \in O(n)$. (If we choose $c = 1$ and $n_0 = 1$, then $T(n) = n - 1 \leq 1 \cdot n$ when $n \geq 1$.)
- If $n=1$ then $T(n)=n-1 \leq 1 \cdot n \rightarrow 1-1 \leq 1$
- If $n=2$ then $T(n)=n-1 \leq 1 \cdot n \rightarrow 2-1 \leq 2$
- If $n=3$ then $T(n)=n-1 \leq 1 \cdot n \rightarrow 3-1 \leq 3$
-
- An algorithm with $T(n) \in O(n)$ is said to have **linear time complexity**.

From slide number 40: We write $T(n) \in O(f(n))$, and say that $T(n)$ has order of $f(n)$, if there are positive constants M and n_0 such that $T(n) \leq M \cdot f(n)$ for all $n \geq n_0$.

Question 10 :calculate the time complexity

- $T(n) = n^2/2 - n/2.$

Answer Question 10

- $T(n) = n^2/2 - n/2$. With Big O notation, this becomes $T(n) \in O(n^2)$, and we say that the algorithm has **quadratic time complexity**.

Question 10 :calculate the time complexity

- $T(n) = n^2/2 - n/2$. With Big O notation, this becomes $T(n) \in O(n^2)$, and we say that the algorithm has **quadratic time complexity**.

$$T(n) = \frac{n^2}{2} - \frac{n}{2} \longrightarrow T(n) = \frac{1}{2}(n^2 - n)$$

Analyze growth for Big-O notation

In Big-O notation, we consider **dominant term** as $n \rightarrow \infty$.

Here, n^2 dominates n , so we can ignore $-n$ and the constant $1/2$.

$$T(n) \in O(n^2)$$

The algorithm has **quadratic time complexity**, $O(n^2)$

Question 11

- Prove that the function $T(n) = 3n^3 + 20n^2 + 5$ belongs to $O(n^3)$.
- Find appropriate constants $c > 0$ and $n_0 \geq 1$ such that:
- $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for all $n \geq n_0$.
- Show your calculation and justification.

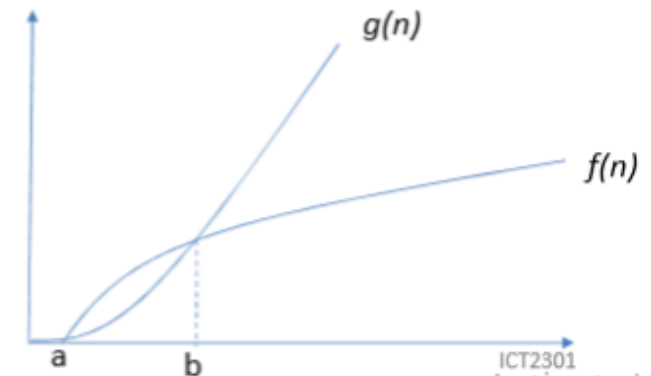
Answer : Question 11

- $T(n) = 3n^3 + 20n^2 + 5 \in O(n^3)$
- We need constants $c > 0$ and $n_0 \geq 1$ such that
- $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for all $n \geq n_0$.
- **(use the values suggested):** choose $c = 4$ and $n_0 = 21$.
- $3n^3 + 20n^2 + 5$ is $O(n^3)$
need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$
this is true for $c = 4$ and $n_0 = 21$

Explanation

$$3 \times 21^3 + 20 \times 21^2 + 5 \leq 4 \times 21^3$$

$$36608 \leq 37044$$



Question 12

- Consider the functions $f(n) = n^3 + 20n^2 + 100n$ and $g(n) = n^3$.
- a) Using Big-O notation, determine whether $f(n) = O(g(n))$.
- (b) If possible, provide constants $c > 0$ and $n_0 \geq 1$ such that the inequality
$$f(n) \leq c \cdot g(n), \text{ for all } n \geq n_0$$
holds.
- (c) Verify your choice of constants by substituting them into the inequality.
- **hint:** You can choose $c = 121$ and $n_0 = 1$.

Answer question 12

(a) Determine whether $f(n) = O(g(n))$

By definition, $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$f(n) = n^3 + 20n^2 + 100n$, $g(n) = n^3$

As n grows large, the n^3 term dominates $20n^2 + 100n$. So, $f(n)$ grows no faster than a constant multiple of $g(n)$.

Therefore, $f(n) = O(n^3)$.

Answer question 12

b) Find constants $c > 0$ and $n_0 \geq 1$

We need $f(n) \leq c \cdot g(n)$ for $n \geq n_0$:

$$n^3 + 20n^2 + 100n \leq c \cdot n^3$$

Divide both sides by n^3 (for $n > 0$):

$$1 + \frac{20}{n} + \frac{100}{n^2} \leq c$$

Now check for $n = 1$ (the smallest integer $n_0 \geq 1$):

$$1 + \frac{20}{1} + \frac{100}{1^2} = 1 + 20 + 100 = 121$$

So, choosing $c=121, n_0=1$ satisfies the condition.

Answer question 12

(c) Verify the inequality with these constants

$$f(n)=n^3+20n^2+100n\leq 121n^3 \text{ for all } n\geq 1$$

Divide both sides by n^3 :

$$1 + \frac{20}{n} + \frac{100}{n^2} \leq 121$$

For $n \geq 1$:

$$\frac{20}{n} \leq 20, \quad \frac{100}{n^2} \leq 100 \quad \Rightarrow \quad 1 + \frac{20}{n} + \frac{100}{n^2} \leq 121$$

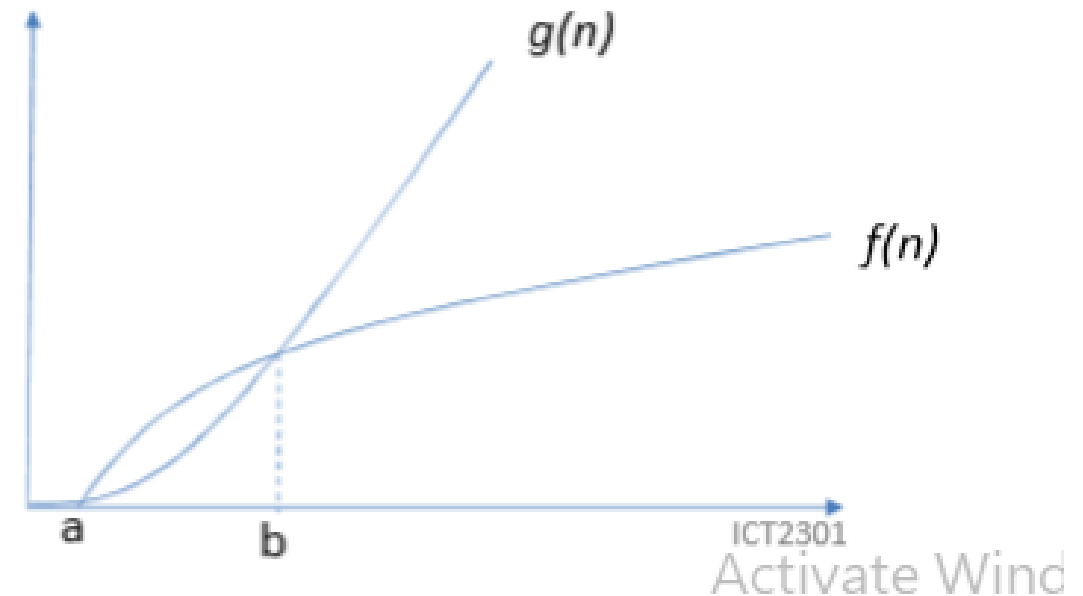
Inequality holds for all $n \geq 1$

Answer question 12

- $f(n)=n^3+20n^2+100n$
- $g(n)=n^3$
- Choosing $c=121$ and $n_0=1$
- $f(n)=O(n^3)$

$$n^3+20n^2+100n \leq C(n^3)$$

$$1+20+100 \leq 121$$



Question 13 :More Big-Oh questions

- Prove that $100n + 5 = O(n)$

Answer – Question 13

- Prove that $100n + 5 = O(n)$

It is necessary to find constants $c > 0$ and $n_0 \geq 1$ such that

$$100n + 5 \leq cn \quad \text{for all } n \geq n_0$$

$$100n + 5 \leq cn \Rightarrow 100 + 5/n \leq c$$

The big-oh condition holds for

$$\begin{aligned} n_0 = 5 \text{ and } c = 101 & \text{ or} \\ n_0 = 1 \text{ and } c = 105 & \text{ (is also a solution)} \end{aligned}$$

$$100n + 5 \leq C(n)$$

$$n_0 = 1 \text{ and } c = 105$$

$100 \times 1 + 5 \leq 105$ is also a solution

Or

$$100n + 5 \leq C(n)$$

$$n_0 = 5 \text{ and } c = 101$$

$$100 \times 5 + 5 \leq 101 \times 5$$

$$505 \leq 505$$

Question 14

$$3n^2 - 100n + 6 = O(n^2) \quad \text{since for } c=3, \quad 3n^2 > 3n^2 - 100n + 6$$

Answer- question 14

- $3n^2-100n+6 = O(n^2)$
- $3n^2-100n+6 \leq C (n^2)$
- $C=3$
- $3n^2-100n+6 \leq 3 (n^2)$
- If $n=1$
- $3x(1^2)-100x1+6 \leq 3x(1^2)$ true
- If $n=2$
- $3x(2^2) -100x2+6 \leq 3x(2^2)$ true

QUESTION 15

- Write pseudocode and a C program to search for an element in an array using linear search.

Linear Search Algorithm

- Simple pseudocode

```
LinearSearch(arr, target):  
    for i from 0 to length(arr) -  
    1:  
        if arr[i] == target:  
            return i // found at  
index i  
    return -1 // not found
```

C code

```
#include <stdio.h>  
  
int linearSearch(int arr[], int n, int target) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target) {  
            return i; // Found  
        }  
    }  
    return -1; // Not found  
}  
  
int main() {  
    int arr[] = {4, 2, 7, 1, 3};  
    int n = 5;  
    int target = 1;  
    int result = linearSearch(arr, n, target);  
    if (result != -1)  
        printf("Element found at index %d\n", result);  
    else  
        printf("Element not found\n");  
    return 0;  
}
```

Time Complexity of Linear Search

Case	Complexity	Explanation
Best Case	$O(1)$	The target element is at the first position — only one comparison is needed.
Worst Case	$O(n)$	The target element is at the last position or not present at all — every element must be checked.
Average Case	$O(n)$	On average, about $n/2$ elements will be checked, which is $O(n)$ in Big-O notation.

Question 16

- a. Write (i) recursive (ii) iterative code for the binary search algorithm
- b. Calculate the time complexity

Binary search Algorithm

- Iterative Method

```
do until the pointers low and high meet each other.  
    mid = (low + high)/2  
    if (x == arr[mid])  
        return mid  
    else if (x > arr[mid]) // x is on the right side  
        low = mid + 1  
    else // x is on the left side  
        high = mid - 1
```

- Recursive

```
binarySearch(arr, x, low, high)  
    if low > high  
        return False  
    else  
        mid = (low + high) / 2  
        if x == arr[mid]  
            return mid  
        else if x > arr[mid] // x is on the right side  
            return binarySearch(arr, x, mid + 1, high)  
        else // x is on the left side  
            return binarySearch(arr, x, low, mid - 1)
```

Binary search Algorithm -Time complexity

- Each iteration cuts the problem size in half:
- After 1st iteration: $n/2$
- After 2nd iteration: $n/4$
- After 3rd iteration: $n/8$
-
-
- After k iterations: $n/2^k$
- we stop $n/2^k=1$
- In **binary search**, you keep halving the input size n .
- After 1 step $\rightarrow$ you have $n/2$ elements left.
- After 2 steps $\rightarrow$ you have $n/4$ elements left.
- After k steps $\rightarrow$ you have $n/2^k$ elements left.
- You continue this process **until there is only 1 element left** to check

$$n/2^k = 1 \implies 2^k = n \implies k = \log_2 n.$$